Project - High Level Design

on

Energymind

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Srashti Bhadoriya	EN22CS301982
2.	Riya Patidar	EN22CS301813
3.	Roshni Patidar	EN22CS301825
4.	Ritik Patidar	EN22CS301809
5.	Pranjal Sharma	EN22CS301731

Group Name: 10D7

Project Number: DO-33

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Shyam Patel Academic

Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	3
Chapter 2	System Design	4-7
	2.1. Application Design	4
	2.2. Process Flow	4
	2.3. Information Flow	5
	2.4. Components Design	6
	2.5. Key Design Considerations	6
	2.6. API Catalogue	7
Chapter 3	Data Design	8-9
	3.1. Data Model	8
	3.2. Data Access Mechanism	8
	3.3. Data Retention Policies	9
	3.4. Data Migration	9
Chapter 4	Interfaces	10
Chapter 5	State and Session Management	11
Chapter 6	Caching	12

Chapter 7	Non-Functional Requirements	13
	7.1. Security Aspects	13
	7.2. Performance Aspects	13
Chapter 8	References	14

1. Introduction

1.1. Scope of the Document

This document provides an exhaustive High-Level Design (HLD) for EnergyMind AI Agents, a state-of-the-art platform that leverages artificial intelligence to transform energy sector research. The scope encompasses the system's architecture, including its multi-agent orchestration via LangGraph, containerization strategies with Docker, and its resilient deployment within a Kubernetes (K3s) ecosystem on AWS. This blueprint serves as BOTH a technical implementation guide and an academic report for university evaluation.

1.2. Intended Audience

Faculty & Academic Mentors: To review the project's adherence to engineering standards and technical innovation.

Energy Market Analysts: To understand how the platform automates data synthesis and report generation.

DevOps & Cloud Engineers: To evaluate the robustness of the CI/CD pipeline and Kubernetes orchestration.

1.3. System Overview

The global energy sector is characterized by immense data volatility and fragmented information. EnergyMind AI Agents addresses this by providing an autonomous research engine capable of real-time market analysis. Unlike traditional search engines, our system utilizes a "Agentic Workflow" where specialized AI agents (Gatekeeper, Researcher, Analyst, Writer, and Reviewer) work collaboratively to:

- Synthesize fragmented data from news, reports, and real-time feeds.
- Verify technical accuracy to eliminate common LLM hallucinations.
- Generate actionable intelligence formatted for executive decision-making.

The system's infrastructure is built for reliability, using a fully automated GitHub Actions pipeline to deploy to a K3s (Kubernetes) cluster on AWS EC2, ensuring that the latest research capabilities are always available with zero-downtime updates.

2. System Design

2.1. Application Design

The system follows a micro-services inspired design:

Frontend Dashboard: Built with Streamlit for a rich, interactive UI.

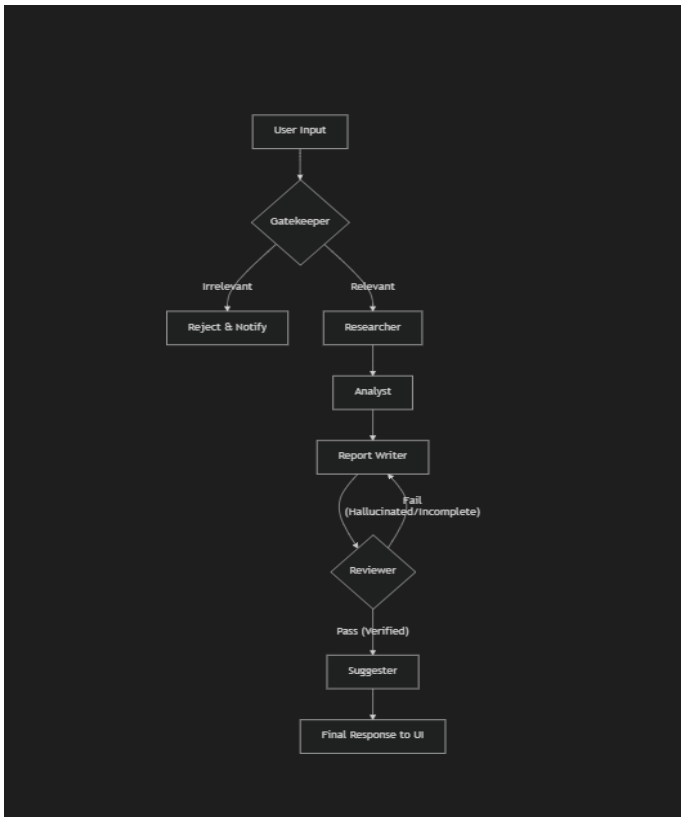
Backend API: FastAPI handles asynchronous requests and agent orchestration.

Intelligence Layer: LangGraph manages the research state machine.

Infrastructure: Docker containers deployed on a Kubernetes (K3s) cluster.

2.2. Process Flow

The research process is governed by a state-aware agentic graph.



Gatekeeper Agent: Uses a zero-shot classification prompt to filter non-energy queries.

Research Agent: Parallelizes search queries across global energy news and academic sources using Tavily.

Analyst Agent: Performs data extraction, identifying specific metrics (e.g., MWh, Carbon Intensity).

Report Writer: Synthesizes structured Markdown from raw analysis.

Reviewer Loop: A critical safety layer that compares the final report against the researcher's raw data to prevent LLM hallucinations.

2.2 Information Flow (CI/CD Pipeline):

Local Dev: Code committed to GitHub main branch.

CI Job (GitHub Actions): Builds Docker image and pushes to Docker Hub.

CD Job: SSHes into AWS EC2, pulls the latest image, and triggers a K3s rolling update.

Runtime: K3s manages pods and exposes ports via NodePort (30000/30001).

2.3. Components Design

2.4.1. Docker Configuration

- **Base Image:** python:3.10-slim.

2.4. Components Design (Technical Specification)

2.4.1. Docker Image Strategy

- **Image Name**: pranjal1712/energymind-app:latest
- **Optimization**: Uses multi-stage builds and .dockerignore to keep image size under 500MB without sacrificing security or performance.
- **Runtime Management**: A

start.sh script coordinates the background startup of the Uvicorn server (FastAPI) and the Streamlit server.

2.4.2. Kubernetes (K3s) Orchestration

- **Deployment**: Specifically configured for LLM workloads with a **1Gi memory limit** to handle large token processing without OOM (Out Of Memory) failures.
- **Service (NodePort)**:
  - **Port 30001**: Exposes the Streamlit UI to the public internet.
  - **Port 30000**: Exposes the FastAPI backend for frontend-to-backend communication.
- **State Persistence**: Uses hostPath volumes on the EC2 instance to ensure

users.db persists across pod restarts.

2.5 Key Design Considerations

Fault Tolerance: K3s automatically restarts containers if they crash.

Zero-Downtime Deployment: GitHub Actions forces a rollout restart, ensuring old pods are only killed after new ones are healthy.

Scalability: The system can be migrated to EKS (Elastic Kubernetes Service) with zero changes to the manifest files.

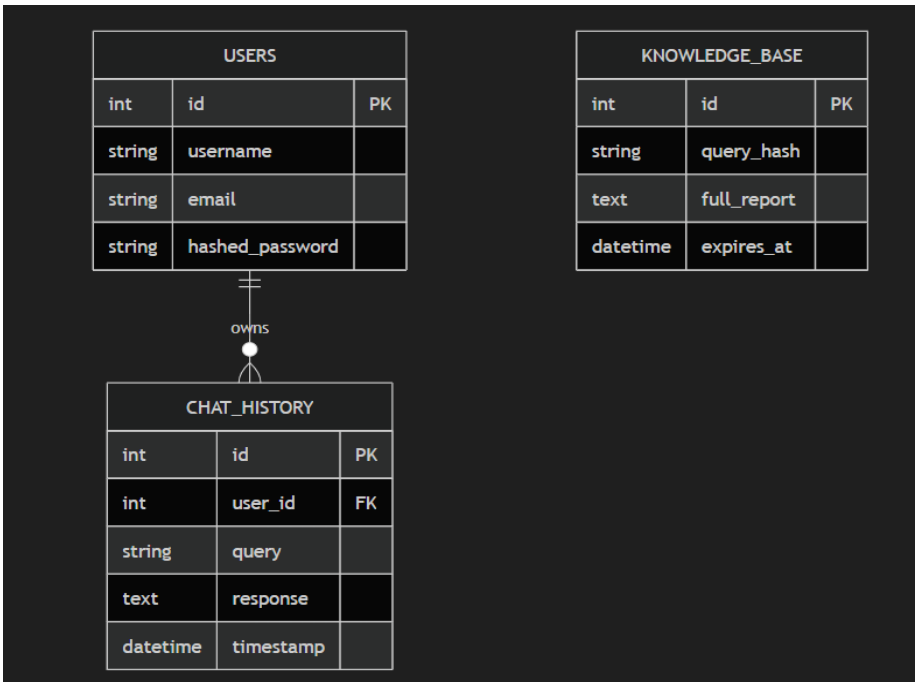
2.4. API Catalogue

GET /history: Retrieves previous research reports.

3. Data Design

3.1. Data Model

The system uses a lightweight yet robust relational schema for persistence.



- **User Schema:** Securely stores hashed credentials.
- **Chat Schema:** Enables short-term and long-term memory for the agents.
- **Knowledge Base:** Optimizes performance by caching expensive LLM results.

3.2. Data Access Mechanism

- **ORM:** SQLAlchemy handles SQLite interactions.
- **Context Management**: Scoped sessions for secure multi-user access.

3.3. Data Retention Policies

- Chat history is stored indefinitely.
- Research results are cached for 24 hours to reduce API costs.

4. Interfaces

- **Web UI:** Streamlit provides a "Glassmorphism" styled dashboard.
- **CLI:** Standard kubectl and git interfaces for management.
- **API Docs**: Interactive Swagger documentation at /docs.

5. State and Session Management

Stateless API: JWT tokens used for authentication.

In-Memory State: LangGraph's

manages research progress across nodes.

6. Caching

Research Cache: SQLite table

knowledge_base

stores previous results.

Layer Cache: Docker image builds optimized with multi-stage logic.

7. Non-Functional Requirements

7.1. Security Aspects

- **Least Privilege Access:** GitHub Actions uses a dedicated SSH key with restricted access to the EC2 instance.
- **Environment Isolation:** No API keys or production secrets are stored in the git repository; they are managed via GitHub Secrets and Kubernetes Secrets.
- **Authentication:** Industry-standard JWT (JSON Web Tokens) with a 24-hour expiration policy for all protected routes.
- **Data Protection:** Passwords are never stored in plain text, utilizing Bcrypt with a salt factor of 12.

7.2. Performance Aspects

- **K3s Resource Limits:** Pods are constrained to 512Mi request / 1Gi limit to prevent resource starvation on the host node.
- **Streamlit Optimization:** The frontend utilizes Streamlit's @st.cache_data to minimize backend round-trips for static assets.
- **Async Concurrency:** The FastAPI backend leverages uvicorn with multiple worker processes to handle high-concurrency research requests.

8. References

- Medicaps University Project Guidelines.
- Kubernetes & Docker Official Documentation.
- LangGraph Orchestration Reference.